Comparative Study Report

RNN, LSTM and GRU for IoT Applications

A Comparative Study on Energy, Weather and Occupancy Sensing

Course

Artificial Intelligence and IoT – CSDX 363

Team

Dwaragesh C (RRN 230191601009)
Nataraj VK (RRN 230191601039)

Programme

B.Tech CSE (IoT)

B.S. Abdur Rahman Crescent Institute of Science & Technology

Abstract

Gated recurrent cells forecast IoT sensor data 0.8–3.5 % more accurately than a plain RNN, at 3–4× the memory and compute. This study implemented **Simple RNN**, **LSTM** and **GRU** models in Keras and applied them to three public IoT datasets: smart-meter energy forecasting (UCI, 2 million readings), weather-station temperature forecasting (Melbourne, 10 years) and smart-building occupancy detection (UCI, 20,560 readings). A fourth experiment extended the energy window from 24 hours to one week.

All models had one 64-unit recurrent layer and identical training settings, repeated over 3–10 random seeds. LSTM was most accurate on energy (RMSE 0.460 kW against 0.471 for the RNN) and on the one-week window, where the RNN got worse while both gated cells improved. GRU was most accurate on temperature (RMSE 2.22 °C) and had the best median occupancy accuracy (97.8 %), with 24 % fewer parameters than the LSTM.

For IoT practice: a plain RNN suits the smallest microcontrollers, an LSTM suits long histories, and a GRU is the best general default.

1. Introduction and Objectives

IoT devices produce time-ordered streams: a smart meter reports every minute, a weather station every day, a building sensor every few seconds. Forecasting or classifying from such a stream needs a model that remembers the recent past. **Recurrent neural networks** do this by carrying a hidden state from one time step to the next.

Three recurrent cells dominate practice: the plain (Elman) RNN, the Long Short-Term Memory (LSTM) and the Gated Recurrent Unit (GRU). They trade accuracy against memory, compute and training time, which is exactly the budget an IoT edge device is short of.

Objectives of this study:

1. Implement RNN, LSTM and GRU models for three real IoT applications.
2. Train all three under an identical budget and compare their test accuracy.
3. Measure the costs that matter on IoT hardware: parameters, model size, compute per inference, latency and training time.
4. Test how each cell copes when the look-back window grows from one day to one week.
5. Recommend which cell suits which IoT scenario.

2. Background: The Three Recurrent Cells

All three cells read one input vector x_t per time step and update a hidden state h_t. They differ in how many gates control that update, which sets both their memory span and their cost. Notation: σ is the sigmoid, ⊙ is element-wise product, W acts on the input and U on the previous state.

2.1 Simple RNN (Elman, 1990)

The plain RNN overwrites its whole state at every step with one tanh layer.

$$h_t = \tanh(Wx_t + Uh_{t-1} + b)$$

During training the error signal is multiplied by U and by tanh′ once per step it travels back. Over long windows this product shrinks towards zero (**vanishing gradient**) or blows up (**exploding gradient**), so the RNN struggles to learn dependencies more than a few dozen steps old.

2.2 LSTM (Hochreiter and Schmidhuber, 1997)

The LSTM adds a separate cell state c_t and three gates. The **forget gate** f_t decides what to keep, the **input gate** i_t what to write, and the **output gate** o_t what to expose.

$$
\begin{aligned}
f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\
o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\
\tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

The cell update is additive, so gradients can flow back through c_t almost unchanged when f_t is near 1. This is what lets an LSTM remember events hundreds of steps old.

2.3 GRU (Cho et al., 2014)

The GRU merges the cell and hidden state and uses two gates: an **update gate** z_t and a **reset gate** r_t. It keeps the additive memory path of the LSTM with one gate fewer.

$$
\begin{aligned}
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) \\
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) \\
\tilde{h}_t &= \tanh\left(W_h x_t + r_t \odot (U_h h_{t-1}) + b_h\right) \\
h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t
\end{aligned}
$$

2.4 Size and Cost of Each Cell

For m input features and n hidden units, the gate count multiplies both the parameters and the multiply-accumulate (MAC) operations per time step.

Cell	Gates	State vectors	Parameters (recurrent layer)	MACs per time step	Parameters at m = 1, n = 64
Simple RNN	0 (one tanh layer)	h	n(m + n) + n	n(m + n)	4,224
LSTM	3 + candidate	h, c	4[n(m + n) + n]	4n(m + n)	16,896
GRU	2 + candidate	h	3[n(m + n) + 2n]	3n(m + n)	12,864

The GRU count uses the Keras default (reset_after = True), which keeps two bias vectors per gate. An LSTM therefore costs about 4× an RNN and a GRU about 3×.

3. IoT Applications and Datasets

The three cells were applied to three public IoT sensor datasets that cover forecasting and classification, single and multi-sensor input, and minute-level to day-level sampling. A fourth experiment reuses the energy data with a week-long window to stress long-range memory.

ID	IoT application	Dataset	Inputs per time step	Sampling	Task	Window	Train / val / test windows
A	Smart energy metering	Individual Household Electric Power Consumption (Sceaux, France, Dec 2006 – Nov 2010, 2,075,259 readings)	9: active and reactive power, voltage, current, 3 sub-meters, hour-of-day sin/cos	1 min, averaged to 1 h	Forecast next-hour active power (kW)	24 h	24,188 / 5,164 / 5,165
B	Weather station	Daily minimum temperatures, Melbourne 1981–1990 (3,650 readings)	1: air temperature	1 day	Forecast next-day minimum temperature (°C)	30 days	2,525 / 517 / 518
C	Smart-building occupancy	Occupancy Detection (office room, Feb 2015, 20,560 readings)	10: temperature, humidity, light, CO2, humidity ratio and the minute-to-minute change of each	1 min	Classify room as occupied or empty	30 min	8,113 / 2,635 / 9,722
D	Long-memory stress test	Same as A	9	1 h	Same as A	168 h (one week)	24,044 / 5,020 / 5,021

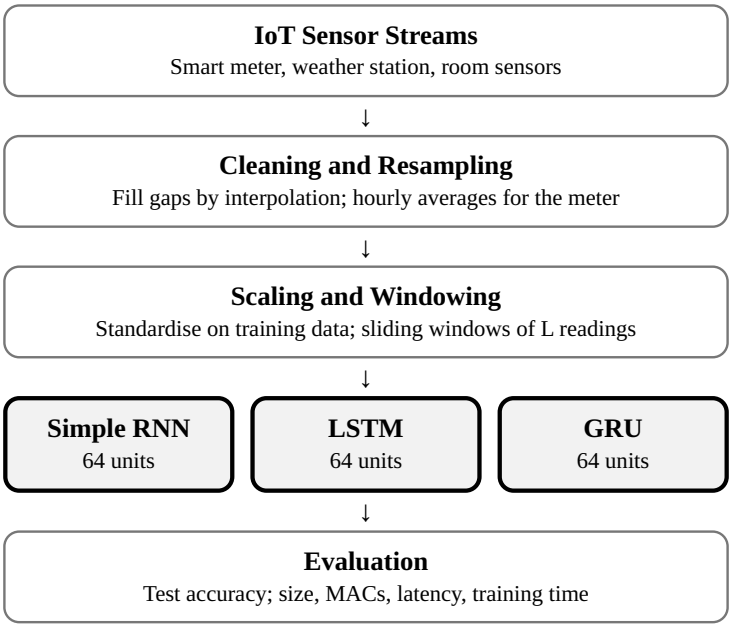
The occupancy labels come from time-stamped photos taken every minute. Its test period (11–18 Feb 2015) is a different week from training (4–10 Feb), so the classifier must generalise across days. About 21 % of test minutes are occupied, so a model that always answers "empty" would already score about 79 % accuracy; **F1 on the occupied class** is the stricter measure.

4. Methodology

Every model saw the same data, the same windows and the same training budget, so differences in the results come from the cell alone.

Experimental pipeline: one data path, three interchangeable cells

The shaded stage is the only one that changes between models



↻ *Repeated over 3 random seeds (10 for occupancy)*

4.1 Pre-processing

1. **Cleaning.** Missing smart-meter readings (1.25 % of rows) were filled by linear interpolation after hourly resampling.
2. **Feature engineering.** The energy task adds sin/cos of the hour of day; the occupancy task adds the first difference of each sensor, since a rising CO2 slope signals people arriving.
3. **Scaling.** Each feature was standardised (zero mean, unit variance) using statistics from the training split only, to avoid leaking test information.
4. **Windowing.** A sliding window of the last L readings forms one input sequence; the target is the next reading (forecasting) or the current occupancy label (classification).
5. **Splitting.** Forecasting data was split chronologically 70 / 15 / 15 % into train, validation and test, never shuffled across time. Occupancy used the dataset's own three recording periods.

4.2 Model Configuration

Setting	Value (identical for RNN, LSTM, GRU)
Architecture	Input (L × m) → one recurrent layer, 64 units → Dense(1)
Output activation	Linear (forecasting), sigmoid (occupancy)
Loss	Mean squared error / binary cross-entropy
Optimiser	Adam, learning rate 0.001
Batch size	64
Epochs	Up to 40, early stopping on validation loss (patience 5, best weights restored)
Repetitions	3 random seeds (0–2), 10 for occupancy (0–9); results reported as mean ± standard deviation

4.3 Evaluation Metrics

- **Forecasting accuracy:** RMSE, MAE and R^2 on the test split, in the original units (kW, °C). A persistence baseline (next value = last value) shows whether a model has learnt anything beyond copying.
- **Classification accuracy:** accuracy and F1-score for the occupied class.
- **Edge cost:** trainable parameters, model size as float32, MACs per inference, training time, and single-window inference latency.

Latency was measured with a plain NumPy re-implementation of each trained network, run one time step at a time like firmware on an edge gateway. The script checks that this re-implementation reproduces the Keras output to within 0.0001 before timing it.

4.4 Environment

Python 3.13, Keras 3.15 on the JAX 0.12 CPU backend, on a 2-vCPU cloud machine with no GPU. Absolute times will differ on other hardware; the ratios between cells are the meaningful part.

5. Results

Gated cells (LSTM, GRU) beat the plain RNN on every forecasting task, but by 0.8–3.5 % RMSE, not by a wide margin. All three cells beat the naive baselines comfortably. Values are mean ± standard deviation over 3 seeds (10 seeds for occupancy); the best value in each column is in bold.

5.1 Application A: Smart Energy Metering (24-hour window)

Model	RMSE (kW)	MAE (kW)	R^2	Epoch of best validation loss
Simple RNN	0.471 ± 0.001	0.333 ± 0.004	0.548 ± 0.003	8
LSTM	**0.460 ± 0.002**	**0.318 ± 0.004**	**0.568 ± 0.005**	7
GRU	0.461 ± 0.002	0.322 ± 0.002	0.567 ± 0.003	10
Persistence baseline	*0.575*	–	–	–

LSTM and GRU are tied within one standard deviation and both are 2.1–2.2 % better than the RNN. All three cut the persistence error by 18–20 %. The remaining error is mostly the unpredictable timing of appliance use, which no model can see coming.

5.2 Application B: Weather-Station Temperature (30-day window)

Model	RMSE (°C)	MAE (°C)	R^2	Epoch of best validation loss
Simple RNN	2.259 ± 0.009	1.760 ± 0.011	0.647 ± 0.003	5
LSTM	2.241 ± 0.008	1.747 ± 0.008	0.652 ± 0.002	9
GRU	**2.222 ± 0.005**	**1.728 ± 0.003**	**0.658 ± 0.002**	24
Persistence baseline	*2.595*	–	–	–

GRU wins here, 1.6 % better than the RNN, with the smallest spread between seeds. The whole range is only 0.04 °C, far below a typical sensor's ±0.5 °C accuracy, so for a single-sensor station the cheapest cell is good enough.

5.3 Application C: Smart-Building Occupancy (30-minute window)

Model	Accuracy, mean ± sd	Accuracy, median	F1 (occupied), mean ± sd	F1, median	Worst seed, accuracy
Simple RNN	95.2 ± 3.3 %	95.3 %	0.892 ± 0.070	0.887	**90.5 %**
LSTM	93.5 ± 1.7 %	93.4 %	0.857 ± 0.037	0.851	90.2 %
GRU	**96.1 ± 3.6 %**	**97.8 %**	**0.912 ± 0.079**	**0.949**	89.6 %
Always "empty" baseline	*79.2 %*	–	*0*	–	–

GRU has the best mean and median, and half of its runs reach 97.8 % accuracy or more. But the run-to-run spread (±3–4 points) is larger than the gaps between cells, so no cell is a statistically clear winner. Validation loss bottomed out within 6 epochs in 29 of 30 runs: the models fit the training week fast, then drift because the test week's sensor levels differ. LSTM was the most consistent but never reached the top scores, which suggests its extra parameters overfit the single training week.

5.4 Experiment D: One-Week Window (168 hours)

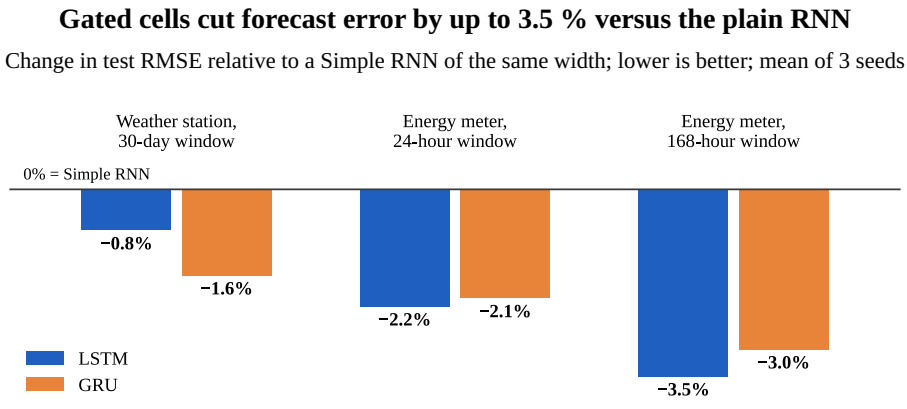
Model	RMSE, 24-h window (kW)	RMSE, 168-h window (kW)	Change from longer window
Simple RNN	0.471	0.474 ± 0.005	+0.7 % (worse)
LSTM	0.460	**0.457 ± 0.002**	−0.7 % (better)
GRU	0.461	0.460 ± 0.003	−0.3 % (better)

Given seven times more history, the gated cells used it and improved slightly, while the plain RNN got slightly worse and less stable between seeds. The gap between LSTM and RNN widens from 2.2 % to 3.5 %. The changes are small, but their direction is what the vanishing-gradient argument in section 2 predicts.

6. Comparative Analysis

The LSTM and GRU buy 1–3.5 % lower error for roughly 3–4× the parameters and compute of a plain RNN. The gain is smallest for one slow sensor and largest when the window is long.

6.1 Accuracy



The gated advantage grows with window length: 2.2 % at 24 hours, 3.5 % at 168 hours. On the short, single-sensor weather series it shrinks to 0.8–1.6 %. On occupancy classification (section 5.3) the ranking was GRU, RNN, LSTM, but within the seed-to-seed noise.

6.2 Edge Cost

Costs are for the energy task (9 inputs, 64 units); the other tasks scale the same way.

Cost	Simple RNN	LSTM	GRU
Trainable parameters	**4,801**	19,009	14,465
Model size, float32	**18.8 KB**	74.3 KB	56.5 KB
Model size if quantised to int8 (estimate)	**4.7 KB**	18.6 KB	14.1 KB
MACs per inference, 24-h window	**112,192**	448,576	336,448
Training time per epoch, 24-h window	**0.60 s**	1.70 s	1.87 s
Total training time, 24-h window	**7.9 s**	20.4 s	27.9 s
Inference latency per window, 24-h (NumPy, 1 core)	**0.16 ms**	0.68 ms	0.75 ms
Total training time, 168-h window	**37 s**	146 s	135 s
Inference latency per window, 168-h	**0.66 ms**	4.13 ms	4.80 ms

The LSTM needs 4.0× the parameters and MACs of the RNN, 2.0–3.9× its training time across the four experiments and 4–6× its latency. The GRU has 24 % fewer parameters and 25 % fewer MACs than the LSTM.

In our measurements the GRU was not faster than the LSTM. The Keras GRU runs two separate matrix products and more element-wise steps per time step, and at batch size 1 with 64 units that per-operation overhead outweighs its smaller MAC count. On a microcontroller, where multiply-accumulates dominate run time, the GRU should come out about 25 % cheaper than the LSTM.

All three models fit easily on an edge gateway such as a Raspberry Pi. On a microcontroller with 32 KB of RAM to spare for weights, only the RNN fits in float32; quantised to int8, all three fit.

6.3 Training Behaviour

The RNN converged fastest in wall-clock time on every task. The GRU took the most epochs on the weather series (best epoch 24, versus 5 for the RNN), learning slowly but steadily. On occupancy all cells overfit within a few epochs, so early stopping mattered more than the choice of cell.

6.4 Overall Scorecard

Criterion	Simple RNN	LSTM	GRU
Accuracy, short windows (24 h, 30 days)	3rd	1st on energy	1st on weather, tied on energy
Accuracy, long window (168 h)	3rd, gets worse	**1st**	2nd
Occupancy classification	2nd	3rd	**1st** (median), not significant

Consistency between seeds	Least consistent on the 168-h window	**Most consistent on occupancy and the 168-h window**	Most consistent on weather, least on occupancy
Memory and compute	**Smallest**	Largest	Middle
Training time (measured)	**Fastest**	Slowest on occupancy and the 168-h window	Slowest on energy (24 h) and weather

7. Discussion: Choosing a Cell for an IoT Deployment

The right cell depends less on the application than on where the model runs and how far back it must look. The gated cells' accuracy edge is real but small, while their cost is a fixed 3–4×.

IoT scenario	Recommended cell	Reason from this study
Battery-powered sensor node (ESP32, ARM Cortex-M), short window, one or two sensors	**Simple RNN**	Within 1–2 % of the best accuracy at a quarter of the memory and compute; only cell that fits 32 KB in float32
Edge gateway forecasting from long histories (a day or more of hourly data)	**LSTM**	Best on the 168-hour window and the cell that gained most from the longer window
Edge gateway or fog node with moderate windows and tight memory	**GRU**	Matches LSTM accuracy with 24 % fewer parameters and 25 % fewer MACs
Multi-sensor event detection (occupancy, fault or anomaly flags)	**GRU**, validated across several days	Best median accuracy and F1, but results vary between training runs, so test on unseen days before trusting it
Cloud analytics with ample compute	**LSTM or GRU**	Accuracy differences are within noise; choose by tooling and training budget

Why the gaps were small

At 24–30 steps the plain RNN's vanishing gradient barely bites, so all three cells see the same useful history. The recent past also carries most of the signal in these series: daily temperature and hourly load depend mostly on the last few readings. The gated cells pull ahead only when distant history matters, which is why their advantage grew in the one-week experiment.

Limitations

- **Fixed hyperparameters.** Every cell used one layer of 64 units with the same settings. Tuning each cell separately could change close rankings.
- **Seed noise.** Differences under about 1 % are within seed-to-seed noise with 3 seeds. Occupancy had only one training week, which drives its high variance.
- **Latency hardware.** Latency was measured in NumPy on a cloud CPU, not on a microcontroller. The GRU's expected 25 % saving over the LSTM on MAC-bound hardware is untested here.
- **One-step forecasts.** Only one-step-ahead forecasts were tested; multi-step horizons may widen the gap.

- **Scope.** Non-recurrent baselines (1-D CNN, temporal convolutional network, Transformer, ARIMA) were outside this comparison.

8. Conclusion and Future Work

RNN, LSTM and GRU models were built and tested on three real IoT applications: smart-meter energy forecasting, weather-station temperature forecasting and smart-building occupancy detection. A fourth experiment stretched the look-back window to one week.

1. **Accuracy.** LSTM and GRU beat the plain RNN on every forecasting task, by 0.8–3.5 % RMSE. All three beat the persistence baseline by 13–20 %.
2. **Memory.** The advantage of gating grows with window length: at 168 hours the RNN got worse while the LSTM and GRU improved.
3. **Cost.** The gated cells need 3–4× the parameters and MACs and 2–7× the training time and latency.
4. **GRU versus LSTM.** The two are statistically tied on energy forecasting. The GRU scored best on weather and occupancy (the latter within noise) with a quarter fewer weights; the LSTM won on the one-week window and was the most consistent.

For most IoT deployments the **GRU is the best default**, with the plain RNN for the smallest microcontrollers and the LSTM for long histories.

Future work: deploy int8-quantised versions on an ESP32 with TensorFlow Lite Micro to measure real latency and energy per inference; test multi-step forecasts; and compare against 1-D CNN and Transformer models.

9. References

1. J. L. Elman, "Finding structure in time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
2. S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
3. K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," *Proc. EMNLP*, 2014.
4. Y. Bengio, P. Simard and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
5. J. Chung, C. Gulcehre, K. Cho and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *NIPS Deep Learning Workshop*, 2014.
6. G. Hebrail and A. Berard, Individual Household Electric Power Consumption [Dataset], UCI Machine Learning Repository, DOI 10.24432/C58K54. https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption
7. L. M. Candanedo and V. Feldheim, "Accurate occupancy detection of an office room from light, temperature, humidity and CO2 measurements using statistical learning models," *Energy and Buildings*, vol. 112, pp. 28–39, 2016. Dataset: https://archive.ics.uci.edu/dataset/357/occupancy+detection
8. Australian Bureau of Meteorology, daily minimum temperatures, Melbourne 1981–1990, as distributed at https://github.com/jbrownlee/Datasets
9. F. Chollet et al., Keras 3, https://keras.io; JAX, https://github.com/jax-ml/jax.

Source code: Appendix A and https://github.com/nagulog1/iot-rnn-lstm-gru (code, results CSV, run logs and figures).

Appendix A: Source Code

The complete code used to produce every result in this report. It is also available at https://github.com/nagulog1/iot-rnn-lstm-gru.

A.1 iot_rnn_lstm_gru.py

Data loading, the three models, training, evaluation and the NumPy edge-latency check for all four experiments.

```
"""
Comparative study: SimpleRNN vs LSTM vs GRU on three IoT applications.

  A. Smart-meter energy forecasting   (UCI Individual Household Electric Power Consumption)
  B. Weather-station temperature      (Daily minimum temperatures, Melbourne 1981-1990)
  C. Smart-building occupancy         (UCI Occupancy Detection: temp, humidity, CO2 sensors)
  D. Long-window stress test          (task A with a 168-hour / one-week look-back)

Every model gets the same budget: one recurrent layer of 64 units + a Dense head,
Adam (lr 1e-3), batch 64, up to 40 epochs, early stopping (patience 5) on validation loss.
Each configuration is trained with 3 random seeds; results are reported as mean +/- std.

Run:  KERAS_BACKEND=jax python3 iot_rnn_lstm_gru.py      (also works with tensorflow / torch backends)
Data: put the three datasets in ./data (download links in the report).
"""
import os, time, json, sys
os.environ.setdefault("KERAS_BACKEND", "jax")
import numpy as np
import pandas as pd
import keras
from keras import layers
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score, accuracy_score, f1_score

DATA = os.path.join(os.path.dirname(os.path.abspath(__file__)), "data")
SEEDS = [int(s) for s in os.environ.get("SEEDS", "0,1,2").split(",")]
UNITS, EPOCHS, BATCH, PATIENCE = 64, 40, 64, 5
CELLS = {"RNN": layers.SimpleRNN, "LSTM": layers.LSTM, "GRU": layers.GRU}


# ---------------------------------------------------------------------------- data
def windows(X, y, lookback, horizon=1):
    """Sliding windows: X[t-lookback:t] -> y[t+horizon-1]."""
    idx = np.arange(lookback, len(X) - horizon + 1)
    Xw = np.stack([X[i - lookback:i] for i in idx]).astype("float32")
    yw = y[idx + horizon - 1].astype("float32")
    return Xw, yw


def chrono_split(n, a=0.70, b=0.85):
    return slice(0, int(n * a)), slice(int(n * a), int(n * b)), slice(int(n * b), n)


def load_energy(lookback):
    df = pd.read_csv(os.path.join(DATA, "household_power_consumption.txt"), sep=";",
                     na_values="?", low_memory=False)
    df.index = pd.to_datetime(df.Date + " " + df.Time, format="%d/%m/%Y %H:%M:%S")
    df = df.drop(columns=["Date", "Time"]).astype("float32")
    df = df.resample("1h").mean().interpolate(limit_direction="both")     # hourly smart-meter feed
    df["hour_sin"] = np.sin(2 * np.pi * df.index.hour / 24)
    df["hour_cos"] = np.cos(2 * np.pi * df.index.hour / 24)
    tr, va, te = chrono_split(len(df))
    sx = StandardScaler().fit(df.iloc[tr]); sy = StandardScaler().fit(df.iloc[tr][["Global_active_power"]])
    X = sx.transform(df); y = sy.transform(df[["Global_active_power"]]).ravel()
    out = {}
    for name, s in zip(["train", "val", "test"], [tr, va, te]):
        out[name] = windows(X[s], y[s], lookback)
    return out, sy, "regression", "kW"
```

```
def load_temperature(lookback=30):
    df = pd.read_csv(os.path.join(DATA, "daily-min-temperatures.csv"))
    v = df.Temp.values.reshape(-1, 1).astype("float32")
    tr, va, te = chrono_split(len(v))
    s = StandardScaler().fit(v[tr]); z = s.transform(v)
    out = {n: windows(z[sl], z[sl].ravel(), lookback) for n, sl in zip(["train", "val", "test"], [tr, va,
te])}
    return out, s, "regression", "degC"


def load_occupancy(lookback=30):
    cols = ["Temperature", "Humidity", "Light", "CO2", "HumidityRatio"]   # all five room sensors
    frames = {n: pd.read_csv(os.path.join(DATA, f)) for n, f in
              [("train", "datatraining.txt"), ("val", "datatest.txt"), ("test", "datatest2.txt")]}
    s = StandardScaler().fit(frames["train"][cols])
    out = {}
    for n, f in frames.items():
        X = s.transform(f[cols]); X = np.hstack([X, np.vstack([np.zeros((1, len(cols))), np.diff(X,
axis=0)])])
        out[n] = windows(X, f.Occupancy.values, lookback)
    return out, None, "classification", ""


# ---------------------------------------------------------------------------- model
def build(cell, input_shape, task):
    m = keras.Sequential([keras.Input(input_shape), CELLS[cell](UNITS),
                          layers.Dense(1, activation="sigmoid" if task == "classification" else None)])
    m.compile(keras.optimizers.Adam(1e-3),
              loss="binary_crossentropy" if task == "classification" else "mse")
    return m


def macs_per_inference(cell, n_in, steps):
    gates = {"RNN": 1, "LSTM": 4, "GRU": 3}[cell]
    return steps * gates * UNITS * (n_in + UNITS) + UNITS


def latency_ms(model, x1, reps=200):
    f = model.predict_on_batch
    for _ in range(20): f(x1)
    t = []
    for _ in range(reps):
        t0 = time.perf_counter(); f(x1); t.append(time.perf_counter() - t0)
    return 1000 * float(np.median(t))


def sigmoid(x): return 1 / (1 + np.exp(-x))


def numpy_forward(model, cell, x):
    """Plain-NumPy forward pass of the trained network for ONE window (T, F) —
    the same step-by-step loop an edge gateway / microcontroller would run."""
    rec, dense = model.layers[0], model.layers[1]
    W = [np.asarray(w) for w in rec.get_weights()]
    h = np.zeros(UNITS, dtype="float32"); c = np.zeros(UNITS, dtype="float32")
    for xt in x:
        if cell == "RNN":
            h = np.tanh(xt @ W[0] + h @ W[1] + W[2])
        elif cell == "LSTM":
            z = xt @ W[0] + h @ W[1] + W[2]
            i, f, g, o = np.split(z, 4)
            c = sigmoid(f) * c + sigmoid(i) * np.tanh(g)
            h = sigmoid(o) * np.tanh(c)
        else:  # GRU, Keras default reset_after=True, gate order (z, r, h)
            xz, xr, xh = np.split(xt @ W[0] + W[2][0], 3)
            hz, hr, hh = np.split(h @ W[1] + W[2][1], 3)
            zg, rg = sigmoid(xz + hz), sigmoid(xr + hr)
            hcand = np.tanh(xh + rg * hh)
            h = zg * h + (1 - zg) * hcand
    Wd, bd = [np.asarray(w) for w in dense.get_weights()]
```

```
    out = h @ Wd + bd
    return sigmoid(out) if dense.activation.__name__ == "sigmoid" else out


def numpy_latency_ms(model, cell, x, reps=300):
    for _ in range(10): numpy_forward(model, cell, x)
    t = []
    for _ in range(reps):
        t0 = time.perf_counter(); numpy_forward(model, cell, x); t.append(time.perf_counter() - t0)
    return 1000 * float(np.median(t))


def run(app, data, scaler, task, unit, cell, seed):
    keras.utils.set_random_seed(seed)
    (Xtr, ytr), (Xva, yva), (Xte, yte) = data["train"], data["val"], data["test"]
    m = build(cell, Xtr.shape[1:], task)
    es = keras.callbacks.EarlyStopping(patience=PATIENCE, restore_best_weights=True)
    t0 = time.perf_counter()
    h = m.fit(Xtr, ytr, validation_data=(Xva, yva), epochs=EPOCHS, batch_size=BATCH,
              callbacks=[es], verbose=0, shuffle=True)
    train_s = time.perf_counter() - t0
    p = m.predict(Xte, batch_size=1024, verbose=0).ravel()
    # the NumPy re-implementation must reproduce Keras exactly, otherwise its timing means nothing
    assert np.allclose(numpy_forward(m, cell, Xte[0]).ravel(), p[:1], atol=1e-4), "numpy forward mismatch"
    r = dict(app=app, model=cell, seed=seed, params=m.count_params(),
             epochs=len(h.history["loss"]), best_epoch=int(np.argmin(h.history["val_loss"])) + 1,
             train_s=train_s, s_per_epoch=train_s / len(h.history["loss"]),
             latency_ms=latency_ms(m, Xte[:1]),
             numpy_latency_ms=numpy_latency_ms(m, cell, Xte[0]),
             macs=macs_per_inference(cell, Xtr.shape[2], Xtr.shape[1]),
             size_kb=m.count_params() * 4 / 1024, n_train=len(Xtr), n_test=len(Xte),
             lookback=Xtr.shape[1], n_features=Xtr.shape[2], unit=unit)
    if task == "regression":
        yt = scaler.inverse_transform(yte.reshape(-1, 1)).ravel()
        yp = scaler.inverse_transform(p.reshape(-1, 1)).ravel()
        r.update(rmse=float(np.sqrt(mean_squared_error(yt, yp))), mae=float(mean_absolute_error(yt, yp)),
                 r2=float(r2_score(yt, yp)))
        # persistence baseline: "next value = last observed value" (feature 0 is the target signal)
        naive = scaler.inverse_transform(Xte[:, -1, 0].reshape(-1, 1)).ravel()
        r["naive_rmse"] = float(np.sqrt(mean_squared_error(yt, naive)))
    else:
        yb = (p > 0.5).astype(int)
        r.update(accuracy=float(accuracy_score(yte, yb)), f1=float(f1_score(yte, yb)))
    r["val_curve"] = [float(v) for v in h.history["val_loss"]]
    return r


if __name__ == "__main__":
    apps = sys.argv[1:] or ["B", "C", "A", "D"]
    loaders = {"A": lambda: load_energy(24), "B": load_temperature, "C": load_occupancy,
               "D": lambda: load_energy(168)}
    out_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "results", "results.jsonl")
    os.makedirs(os.path.dirname(out_path), exist_ok=True)
    for app in apps:
        data, scaler, task, unit = loaders[app]()
        print(app, {k: v[0].shape for k, v in data.items()}, flush=True)
        for cell in CELLS:
            for seed in SEEDS:
                r = run(app, data, scaler, task, unit, cell, seed)
                with open(out_path, "a") as f: f.write(json.dumps(r) + "\n")
                print({k: (round(v, 4) if isinstance(v, float) else v) for k, v in r.items() if k !=
"val_curve"}, flush=True)
```

A.2 summarise.py

Averages the individual runs into the mean ± standard deviation values used in the result tables.

```
"""Aggregate results.jsonl -> results_summary.csv (mean and std over seeds)."""
import os
HERE = os.path.dirname(os.path.abspath(__file__))
```

```
import json, pandas as pd
rows = [json.loads(l) for l in open(os.path.join(HERE, "results", "results.jsonl"))]
df = pd.DataFrame(rows).drop(columns=["val_curve"])
metrics = [c for c in ["rmse", "mae", "r2", "naive_rmse", "accuracy", "f1", "train_s", "s_per_epoch",
                       "epochs", "best_epoch", "latency_ms", "numpy_latency_ms"] if c in df]
g = df.groupby(["app", "model"], sort=False)
summ = g[metrics].agg(["mean", "std"])
summ.columns = [f"{a}_{b}" for a, b in summ.columns]
fixed = g[["params", "size_kb", "macs", "lookback", "n_features", "n_train", "n_test"]].first()
out = fixed.join(summ).reset_index()
out.to_csv(os.path.join(HERE, "results", "results_summary.csv"), index=False)
pd.set_option("display.width", 250, "display.max_columns", 50)
for app, d in out.groupby("app", sort=False):
    cols = ["model", "params", "size_kb", "macs"] + [c for c in d if any(c.startswith(m) for m in
            ["rmse", "mae", "r2", "naive", "accuracy", "f1", "train_s", "s_per", "epochs", "best",
"numpy"])]
    print(f"\n=== {app}"); print(d[cols].dropna(axis=1, how="all").round(4).to_string(index=False))
```

A.3 download_data.sh

Downloads the three public datasets.

```
#!/usr/bin/env bash
# Downloads the three public IoT datasets used in this study into ./data
set -euo pipefail
cd "$(dirname "$0")"
mkdir -p data && cd data

# A / D: household smart-meter readings (UCI), mirrored on GitHub
curl -fSL -o household_power_consumption.zip \
  https://raw.githubusercontent.com/jbrownlee/Datasets/master/household_power_consumption.zip
unzip -o -q household_power_consumption.zip && rm household_power_consumption.zip

# B: daily minimum temperatures, Melbourne 1981-1990
curl -fSL -o daily-min-temperatures.csv \
  https://raw.githubusercontent.com/jbrownlee/Datasets/master/daily-min-temperatures.csv

# C: office-room occupancy detection (UCI), author's GitHub copy
for f in datatraining.txt datatest.txt datatest2.txt; do
  curl -fSL -o "$f" "https://raw.githubusercontent.com/LuisM78/Occupancy-detection-data/master/$f"
done

ls -la
```

A.4 requirements.txt

Python packages needed to run the code.

```
keras>=3.15
jax[cpu]>=0.12
numpy>=2.0
pandas>=2.2
scikit-learn>=1.5
```